

QML Implementation Comparison

Architecture and Paradigm Divergences: Arjun Approach vs. Lawun Approach

Strategic Overview: Both frameworks share a common underlying architecture utilizing mixed-state devices, PyTorch backpropagation, and identical conceptual objective structures. However, critical variations in circuit mapping, prototype memory updates, and inference logic establish two distinct algorithmic tracks rather than a single unified pipeline.

1. Circuit & Prototype Architecture

COMPONENT	ARJUN APPROACH	LAWUN APPROACH
Quantum Circuit Architecture	- Fixed 5 qubits, 3 layers. - Custom encoding: $RY(x[w \% n])$, $RZ(x[(w + 1) \% n])$ per wire. - Ansatz: RY/RZ per qubit + ring CNOT. - Noise model: Depolarizing $p = 0.01$. - Parameter θ shape: (3, 5, 2); initialized with small Gaussian distributions. - Features: StandardScaler $\rightarrow$ PCA(95%) $\rightarrow$ 7 dimensions $\rightarrow$ mapped to angles in $[0, \pi]$.	num_qubits dynamically set to the feature dimension of q8 (no PCA applied). $AngleEmbedding(Y) + StronglyEntanglingLayers$. 2 layers; identical depolarizing noise $p = 0.01$. - Parameter θ shape derived from $StronglyEntanglingLayers.shape$; initialized to zeros. - Returns (z, ρ) directly from the QNode.

Architectural Mismatch: These represents entirely different Hilbert-space maps. Consensus on the "MAQT" framework remains purely architectural at the loss-wrapper level until the circuit topologies are standardized. Arjun builds z from density matrix ρ via Pauli-Z traces, whereas Lawun outputs both parameters natively.

Prototypes & Memory Bank	- Class representation: RunningPrototypes. - Bank Update Strategy: Exponential Moving Average (EMA) with $m = 0.9$, computed per batch inside the loss function. - Final State: Finalized bank leverages the last available EMA state.	- Class representation: PrototypeBank. - Bank Update Strategy: Running mean (sum/count), resetting completely at each epoch. - Final State: Evaluates compute_prototypes over the full training set after determining the optimal θ.
--------------------------	---	--

Architectural Mismatch: The choice between EMA and epoch-based running means fundamentally shifts the embedding geometry and subsequent conformal scores. A uniform approach must be selected to baseline the Algorithm-1 memory architecture.

2. Loss Function Details

$$L = L_{CE} + \lambda_1 L_{intra} + \lambda_2 L_{inter} \quad | \quad L_{intra} \propto (1 - F) \quad | \quad L_{inter} = -D_{tr}$$

LOSS COMPONENT	ARJUN APPROACH	LAWUN APPROACH
Classification Loss (L_{CE})	Focal Loss with parameter $\gamma = 2$.	Plain Cross Entropy combined with tempered class weights.
Intra-class Loss (L_{intra})	Mean evaluated directly over all samples.	Mean compiled from individual per-class means.
Hyperparameters (λ_1 / λ_2)	Defaults: 0.5 / 0.1, paired with a warmup curriculum and grid sweep optimization.	Fixed constraints: 0.5 / 0.3.
Fidelity & Distance Metrics	Clamped PennyLane Fidelity (F) and Trace Distance (TD).	Standard PennyLane Fidelity (F) and Trace Distance (TD) without bounding/clamping.

3. Inference & Zero-Day Detection (Algo 2–3)

METRIC / PIPELINE	ARJUN APPROACH	LAWUN APPROACH
Conformal Prediction	Data pipelines are prepared; however, execution logic is omitted.	Non-conformity score defined as $s = 1 - \max(F)$ with significance level $\alpha = 0.05$ quantile.
Prediction Logic	Evaluates classification head argmax exclusively.	Computes head evaluation and quantum fidelity; triggers a rejection status if $s > q + \text{radius}$.
Zero-day Capability	Data structure is initialized and loaded, but remains unutilized during predictive inference.	Actively measures and outputs the evaluation detection rate on the <code>q8_zeroday</code> dataset.

Architectural Mismatch: For implementations prioritizing certified zero-day discovery, the Lawun framework serves as the primary technical reference; the Arjun implementation is functionally incomplete in this domain.

4. Data & Training Protocol

PROTOCOL VARIABLE	ARJUN APPROACH	LAWUN APPROACH
Class Configuration	Top-6 classes selected, capped at 300 samples per class.	Natural multiclass label structures imported directly from parquet data.
Data Split Strategy	Discrete, pre-allocated files for training, testing, calibration, and zero-day subsets.	Validation, calibration, and testing are designated as stratified subsets drawn from training data (50 samples each).
Batch & Epoch Sizing	16 batches / 10 epochs (includes a retraining phase post hyperparameter sweep).	64 batches / 30 epochs, regulated by early stopping criteria based on validation macro-F1 score.
Imbalance Mitigation	Utilizes a PyTorch WeightedRandomSampler.	Implements class-weighted Cross Entropy loss natively.

5. Established Alignments

The following core properties are fully synchronized between both approaches, confirming that the codebases are distinct structural forks of the same fundamental blueprint:

- **Execution Layer:** Both frameworks leverage a mixed-state simulator device, written in PyTorch, using gradient backpropagation.
- **Noise Execution:** Application of depolarizing noise occurs immediately following the quantum layer operations.
- **Classification Head:** A linear layer maps Pauli-Z expectations to target dimensions during Cross-Entropy optimization.
- **Geometric Metrics:** Quantum prototype layouts are calculated exclusively using quantum state fidelity and trace distance formulas.
- **Objective Paradigm:** The high-level optimization formula follows the same core multi-task learning configuration for Algorithm-1.